

5 adet 7 segment display P0 ve P1 e baęlı

4x3 tuř takımı P2'ye baęlı

2952 satır kod

10 saniyede bir veri alınıyor gün sonunda siliniyor

Gün içinde toplanan veri ; $6 \times 60 \times 24 = 8640$ adet

$8640 = 21C0h$ yapar (16lık sistemde)

Verilerin başlangıç adresi ; 3000h

Bitiş adresi ; $3000h + 21C0h = 51C0h$ (1 çıkarmamız gerekir 3000h'dan başladığı için)

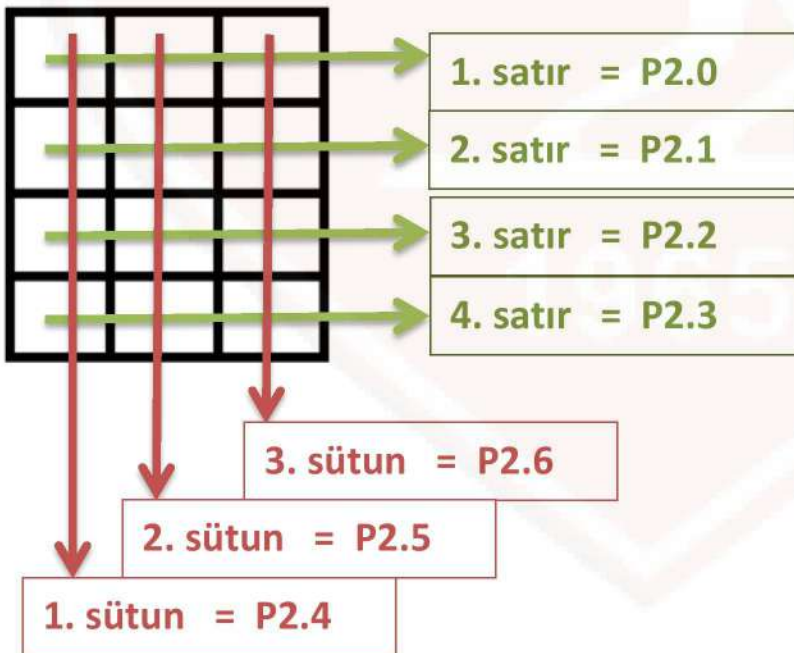
$51C0h - 1 = 51BFh$ olur.

2952 = B88h olur (16lık sistemde)

Programın başlangıç adresi ; 0030h

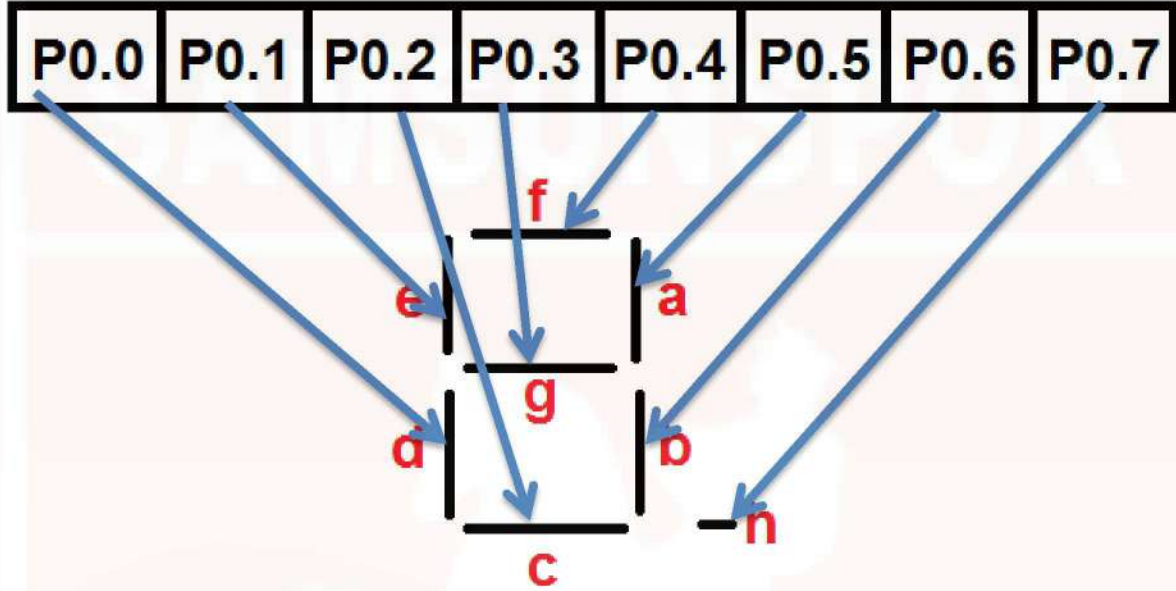
Bitiş adresi ; $0030h + B88h = BB8h$ (1 çıkarmamız gerekir aynı şekilde)

$BB8h - 1 = BB7h$ olur.

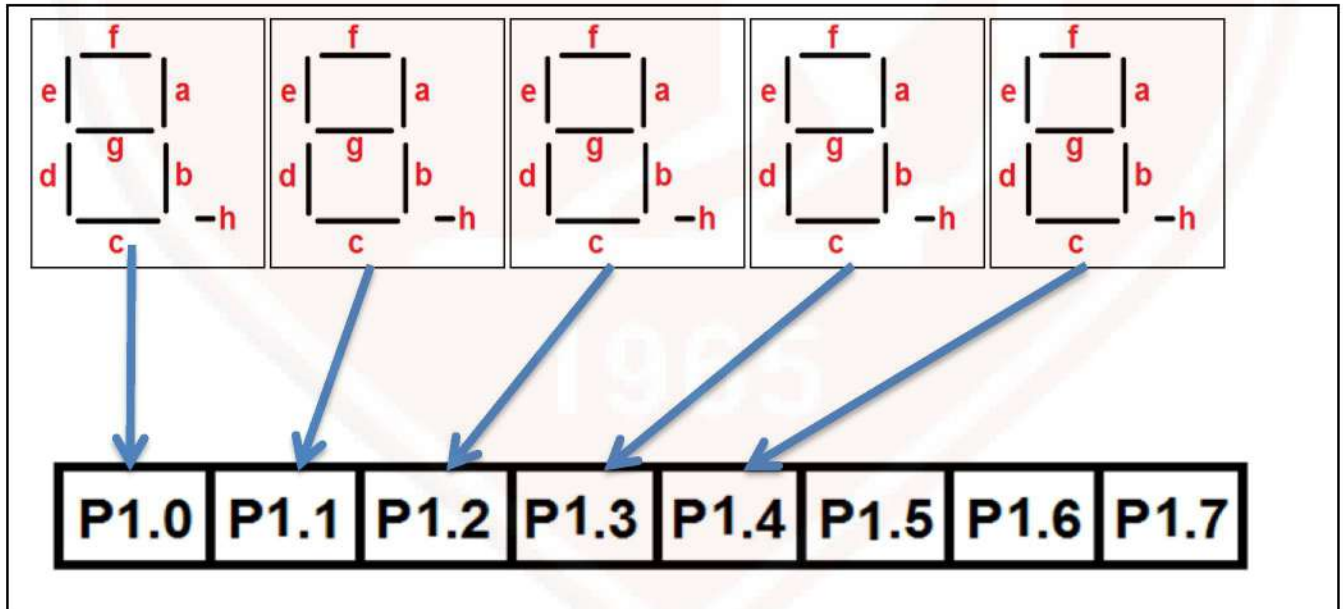


4x3 lük tuř takımı satır ve sütunların her biri Port2'nin birer bitine bağlanmalıdır. Hangi bite hangi satır/sütun bağlandığı önemli değildir. Toplam 7 tane bit kullanıldığı için bir bit (P2.7) boşta kalır.

5 adet 7 segment display için 2 port gerekir. Birisi displayde gözükecek rakam için diğeri de displayleri aktif yapmak içindir. Port0'ı rakam yazdırmak için, Port1'i de yetkilendirme için kullanalım. (tam tersi de olabilir nasıl isterseniz)



Burda rastgele olarak her bite bir parça bağladım.Sıralı olarak da bağlanabilir ;
P0.0 = a ledi P0.2 = c ledi P0.4 = e ledi P0.6 = g ledi
P0.1 = b ledi P0.3 = d ledi P0.5 = f ledi P0.7 = h ledi



Görüldüğü gibi 5 adet 7 segment displayin yetki bitleri Port1e bağlandı. 5 tane olduğu için 3 bit boşa kaldı. Bağlama sırası farketmez istediğiniz gibi bağlayabilirsiniz.

P1.0 = 1. display

P1.2 = 3. display

P1.4 = 5. display

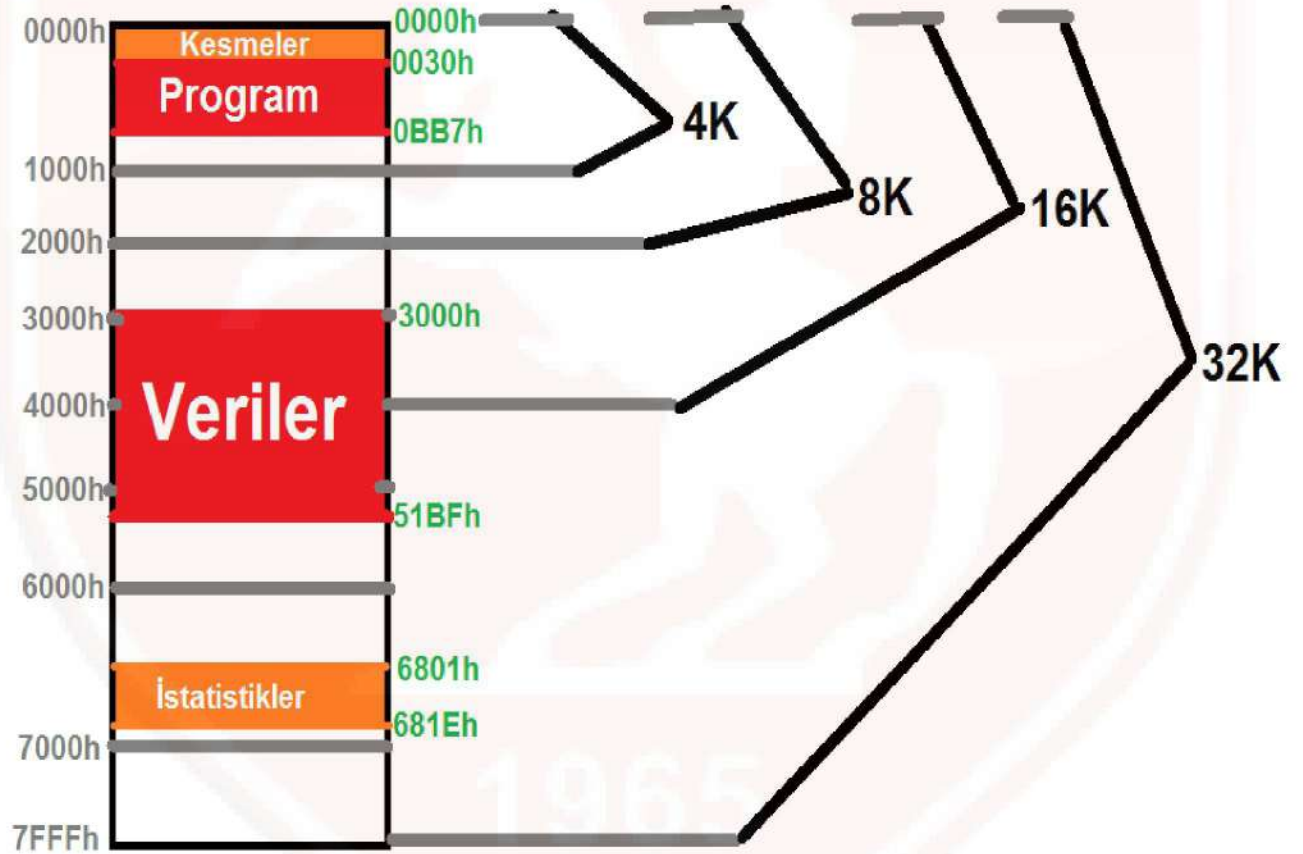
P1.1 = 2. display

P1.3 = 4. display

P1.5, P1.6 ve P1.7 boş

SAMSUNSPOR

Aşağıda kaç KB'lık bellek kullanacağı resim üzerinde gösterilmiştir.



2. SORU

MOV 43h, #00h sayacı sıfırlıyoruz

MOV DPTR,#2000h data pointera dizinin başlangıç adresini yüklüyoruz

MOVB A, @DPTR dizinin ilk elemanını aküye alıyoruz

MOV 40h, A ilk elemanı en büyük kabul ediyoruz(sırayla karşılaştıracağız)

DONGU:

MOVB A, @DPTR (sıradaki elemanı aküye alıyoruz)

CJNE A , 40h , ESITDEGIL (aküyle 40h'ı değeri karşılaştırıyoruz)

INC 43h (eğer eşitse sayacı 1 artırıyoruz)

SJMP KONTROL (döngünün sonuna git)

ESITDEGIL:

JC KONTROL (eğer elde varsa aküdeki sayı küçüktür döngü sonuna git ; elde yoksa devam)

MOV 40h, A (aküdeki sayıyı 40h'a yaz)

MOV 41h, DPH (sayının bulunduğu adresin yüksek kısmını 41ha yaz)

MOV 42h, DPL (sayının bulunduğu adresin düşük kısmını 42ha yaz)

MOV 43h, #01h (sayacı 1 yapıyoruz yeni sayı olduğu için)

KONTROL:

INC DPTR (sonraki eleman için 1 artırıyoruz)

MOV A, DPH (yüksek kısmı kontrol etmek için aküye alıyoruz)

CJNE A,#30h, DONGU (eğer yüksek kısım 30 ise döngü sonlanıyor ; 3000h olursa biter çünkü)

END

ORG 0000h

LJMP MAIN

ORG 0003h ; harici kesme 0 (int0) için kesme vektörü

LJMP INTOKESME

ORG 000Bh ; timer 0 için kesme vektörü

LJMP TOKESME

ORG 0013h ; harici kesme 1 (int1) için kesme vektörü

LJMP INT1KESME

ORG 0030h

MAIN:

MOV IE,#87h ; kesmeler aktif edilir (int0, t0, int1)

MOV TMOD,#81h ; timer0 mod1 modunda kullanılacak

MOV IP,#02h ; timer0 kesmesi varken diğer kesmeler engellenecek

SETB IT0 ; sensör-k lojik 0 da çalıştığı için

SETB IT1 ; sensör-Y lojik 0 da çalıştığı için

CLR P0.1 ; itici-k normal konuma getirildi

CLR P0.2 ; itici-y normal konuma getirildi

SETB P0.0 ; motor çalıştırıldı

SJMP \$; sonsuz döngüye girdi

INTOKESME:

CLR P0.0 ; sensör-K dan kesme geldiğinde motor durdurulacak

MOV TH0, #HIGH(-60 000) ; sayma değerinin yüksek baytı yüklendi

MOV TL0, #LOW (-60 000) ; sayma değerinin düşük baytı yüklendi

SETB TR0 ; timer 0 çalıştırıldı ve saymaya başladı

SETB P0.1 ; itici-k çalıştırıldı ve ilerlemeye başladı

RETI ; kesme alt programından geri dön

INT1KESME:

CLR P0.0 ; sensör-Y den kesme geldiğinde motor durdurulacak

MOV TH0, #HIGH(-60 000) ; sayma değerinin yüksek baytı yüklendi

MOV TL0, #LOW (-60 000) ; sayma değerinin düşük baytı yüklendi

SETB TR0 ; timer 0 çalıştırıldı ve saymaya başladı

SETB P0.2 ; itici-y çalıştırıldı ve ilerlemeye başladı

RETI kesme alt programından geri dön

TOKESME:

CLR P0.1 ; itici-k normal konuma getirildi

CLR P0.2 ; itici-y normal konuma getirildi

CLR TF0 ; timer 0 taşma bayrağı temizlendi

SETB P0.0 ; motor tekrar çalışmaya başladı

RETI ;kesme alt programından geri dön

END